



---

# ***Algorithms, Design, and Analysis***

***Dr. Allam Ameen***

***Assistant Professor***

***Helwan International Technological University (HITU)***

***Electronics Research Institute, Microstrip Department***

**Spring 2026**

# Lecture Outcomes

- Learning how to calculate the efficiency of any algorithm using a random access machine.
- Learning how to calculate the time efficiency- asymptotic notation

# The efficiency of Algorithms – Random Access Machine

## Efficiency of Algorithms

**Efficiency: Amount of resources used by an Algorithm.**

- ❑ An algorithm is considered efficient if its resource consumption is below some acceptable level. It should run in a reasonable amount of time on an available computer or hardware specifications.
- ❑ The two most common used measures are:
  - Time: How long does the algorithm take to complete.
  - Space: How much working memory is needed by the algorithm.  
(This has two aspects: The amount of memory needed by the code, and the amount of memory needed for the data on which the code operates.)

For an algorithm to be efficient, the above two factors should be optimized, i.e., the algorithm should fulfill all the proper requirements with minimum amount of time and within specified space limits and should not overload the memory.

## **Space Efficiency**

- ❑ For large quantities of data space/memory used should be analyzed.**
- ❑ The major components of space are:**
  - instruction space**
  - data space**
  - run-time stack space**

## **Analysis of Time Complexity**

### **❑ Running time depends upon:**

- **Compiler used**
- **R/w speed to Memory and Disk**
- **Machine Architecture : 32-bit vs 64**
- **Input size (rate of growth of time)**

### **❑ When analyzing for time complexity we can take two approaches:**

#### **1. Estimation of running time – Random Access Machine (hypothetical Model Machine)**

**By analysis of the code, we can do:**

- a. Operation counts - select operation(s) that are executed most frequently and determine how many times each is done.**
- b. Step counts - determine the total number of steps, possibly lines of code, executed by the program.**

#### **2. Order of magnitude/asymptotic categorization - gives a general idea of performance.**

## Random-Access Machine (RAM).

- Memory consists of an **infinite array of cells**.
- Each cell can store at most **one data item** (bit, byte, a record, ..).
- Any memory cell can be accessed in **unit time**.
- Instructions are executed **sequentially**.
- All basic instructions take **unit time**:
  - Load/Store
  - Arithmetic's (e.g., +, -, \*, /)
  - Logic (e.g., >, <)

□ Running time of an algorithm is the number of RAM instructions it executes

By analysis of the code, we can do:

- a. **Operation counts** - select operation(s) that are executed most frequently and determine how many times each is done.
- ◊ b. **Step counts** - determine the total number of steps, possibly lines of code, executed by the program.

## RAM Model

- Single Processor
- Sequential Execution
- 1 Time unit / Operation:
- (assignment, +, -, \*, /, logical operations)

```
list_sum (A, n)
```

```
{
```

```
Sum = 0
```

```
for i = 1 to n-1
```

```
    sum = sum + A(i)
```

```
return sum
```

```
}
```

| Cost | # of times |
|------|------------|
| 1    | 1          |
| 2    | n+1        |
| 2    | n          |
| 1    | 1          |



## ❑ RAM model is not realistic:

- **Memory is finite** (even though we often imagine it to be infinite when we program).
- **Not all memory accesses** take the same time (**cache, main memory, disk**).
- **Not all arithmetic operations** take the same time (e.g., multiplications are expensive)

# **The efficiency of Algorithms – Asymptotic Notation**

## Time Efficiency – Asymptotic Notation

❑ In Asymptotic Analysis, the performance of an algorithm is evaluated in terms of input size (we don't measure the actual running time). We calculate, how does the time (or space) taken by an algorithm increases with the input size.

3 cases should be considered:

1. worst case
2. average case
3. best case

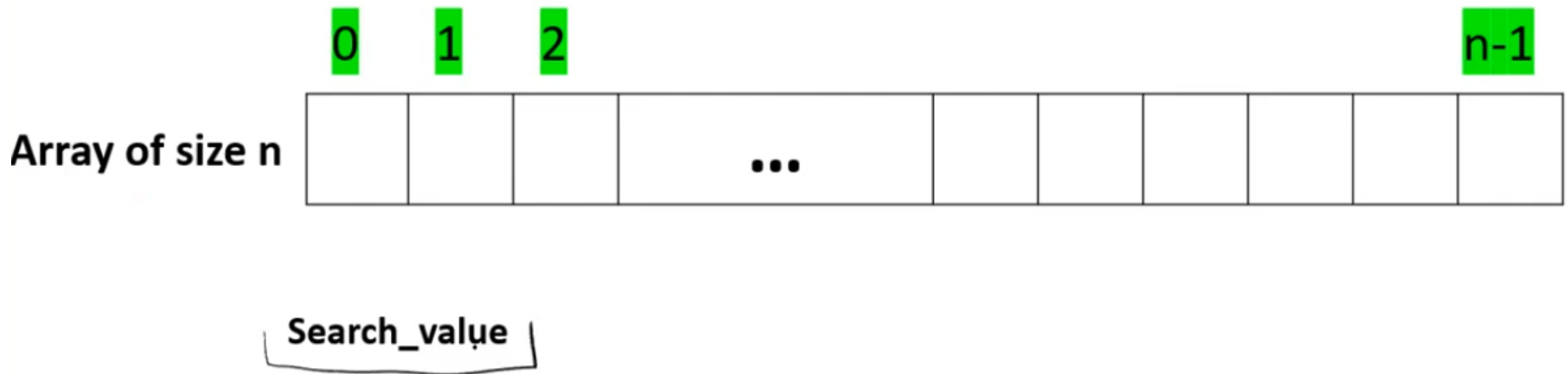
❑ Usually, worst case is considered since it gives an upper bound on the performance that can be expected.

❑ Best case is rarely considered

❑ The average case is usually harder to determine - more data dependent and is often the same as the worst case.

## Time Efficiency – Asymptotic Notation

### Example: Linear Search



# Asymptotic Notation

❑ The most common Asymptotic Notation for calculating Algorithm's complexities are:

➤ **Big-O Notation**

Big-O notation represents the upper bound of the running time of an algorithm. Thus, it gives the worst-case complexity of an algorithm.

➤ **Big- $\Omega$  Notation**

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best-case complexity of an algorithm.

➤ **Big- $\Theta$  Notation**

Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.

## Big-O Notation

□ Given  $f(n)$  and  $g(n)$  - functions defined for positive integers

Then

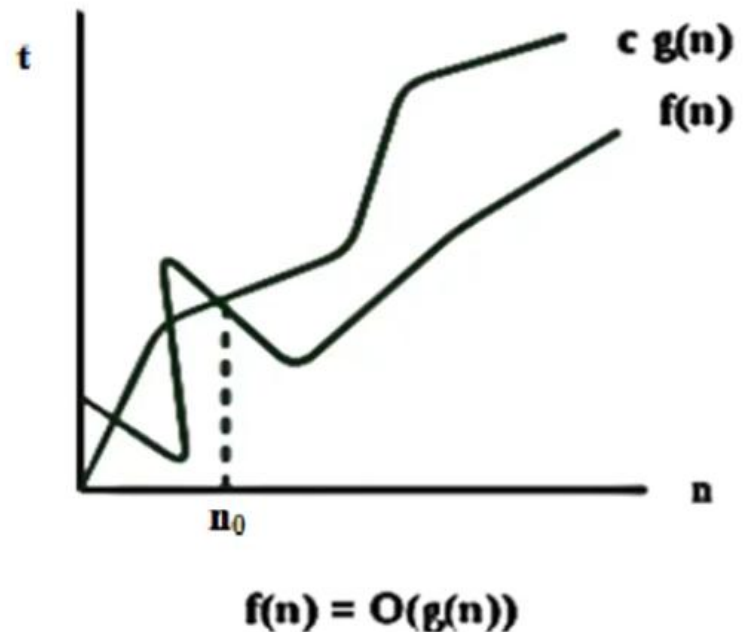
$$f(n) = O(g(n))$$

If there exists a  $c$  ( $c \geq 1$ ) such that:

$$f(n) \leq c g(n)$$

for all sufficiently large positive integers  $n$

$$(n \geq n_0)$$



□ The Big O notation defines an upper bound of an algorithm, it bounds a function only from above.

### Example 1:

$$f(n) = 4n + 6$$

$$O(n)$$

↑

$$5n$$

$O(n)$

$$c \geq 1$$

$$cn > 4n + 6$$

$$5n = 4n + 6$$

$$n = 6$$

$$n_0 = 6 \quad \boxed{5n} \quad \boxed{4n+6}$$

$$\boxed{n_0 > 6}$$

### Example 2:

$$f(n) = 4n^2 + n + 12$$

$$O(n^2)$$

↑

$$c \geq 1$$

$$\boxed{5n^2} \geq \sqrt{4n^2 + n + 12}$$

$$n^2 - n - 12 = 0$$

$$(n-4)(\underline{n+3}) = 0$$

$$\boxed{n=4}$$

$$5n^2$$

80

$$4n^2 + n + 12$$

64 + 4 + 12

$$\boxed{n_0 > 4}$$

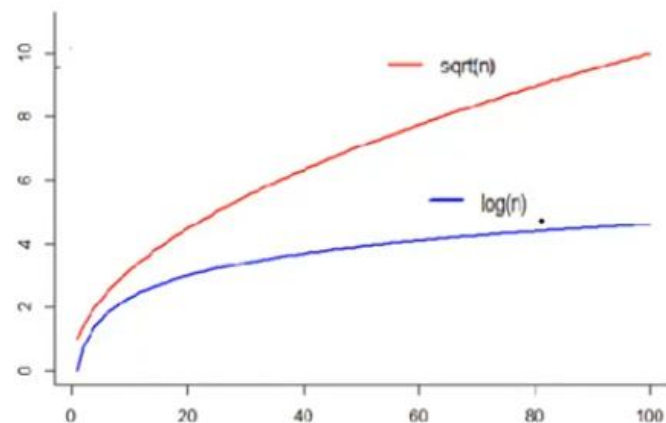
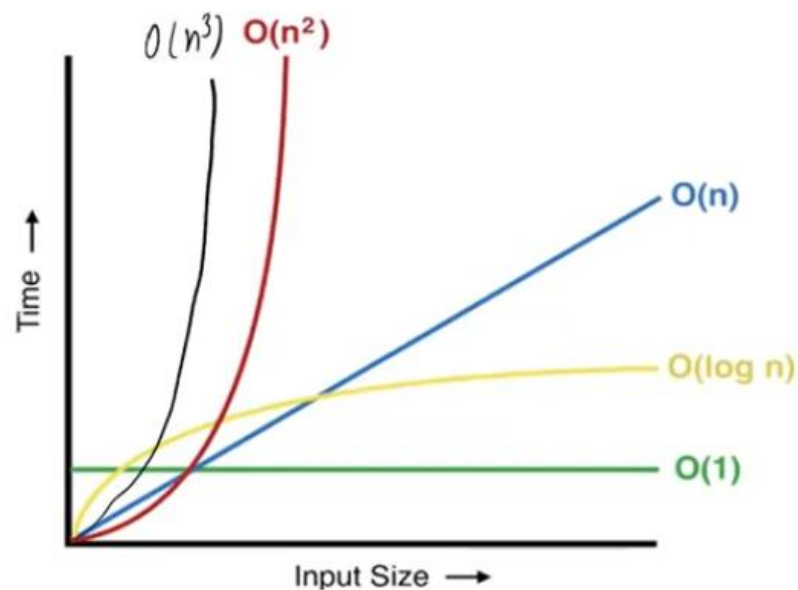
## Most Common Big-O complexities

| Big-O Notation | Name         | Example                                         |
|----------------|--------------|-------------------------------------------------|
| $O(1)$ ✓       | Constant     | Odd or Even number                              |
| $O(\log n)$    | Logarithmic  | Finding element on sorted array with            |
| $O(n)$         | Linear       | Find max element in unsorted array,             |
| $O(n \log n)$  | Linearithmic | Sorting elements in array ( <u>Merge Sort</u> ) |
| $O(n^2)$       | Quadratic    | Most of sorting Algorithms                      |
| $O(n^3)$       | Cubic        | 3 variables equation solver                     |
| $O(2^n)$       | Exponential  | Find all subsets                                |
| $O(n!)$        | Factorial    | Find all permutations of a given set/string     |





| Function   | Big-O         | Name        |
|------------|---------------|-------------|
| $1$        | $O(1)$        | Constant    |
| $\log n$   | $O(\log n)$   | logarithmic |
| $n^{1/2}$  | $O(n^{1/2})$  | Square Root |
| $n$        | $O(n)$        | linear      |
| $n \log n$ | $O(n \log n)$ | $n \log n$  |
| $n^2$      | $O(n^2)$      | quadratic   |
| $n^3$      | $O(n^3)$      | cubic       |
| $2^n$      | $O(2^n)$      | exponential |
| $n^c$      | $O(n^c)$      | Polynomial  |
| $n!$       | $O(n!)$       | factorial   |



## Steps for finding Big-O runtime:

1. Figure out what the input is and what  $n$  represents.
2. Express the maximum number of operations, the algorithm performs in terms of  $n$ .
3. Eliminate all excluding the highest order terms.
4. Remove all the constant factors.

$$f(n) = 3n^3 + n^2 + 15$$

$O(n^3)$

$$f(n) = 4n^2 + 3n + 15$$

$O(n^2)$

## $\Omega$ Notation

Big O notation provides an asymptotic upper bound on a function,

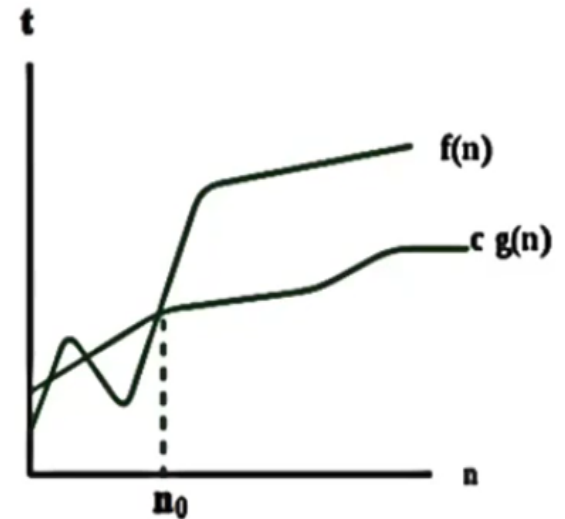
$\Omega$  notation provides an asymptotic lower bound.

$\Omega$  Notation can be useful when we have lower bound on time complexity of an algorithm.

$\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$

$0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0\}$ .

Since the best-case performance of an algorithm is generally not useful, the Omega notation is the least used notation among all three.



$f(n) = \Omega(g(n))$

$$f(n) = 5n + 10$$

$$\Omega(n)$$

## $\Theta$ Notation

- The theta notation bounds a function from above and below, so it defines exact asymptotic behavior.

$\Theta(g(n)) = \{f(n): \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that:}$

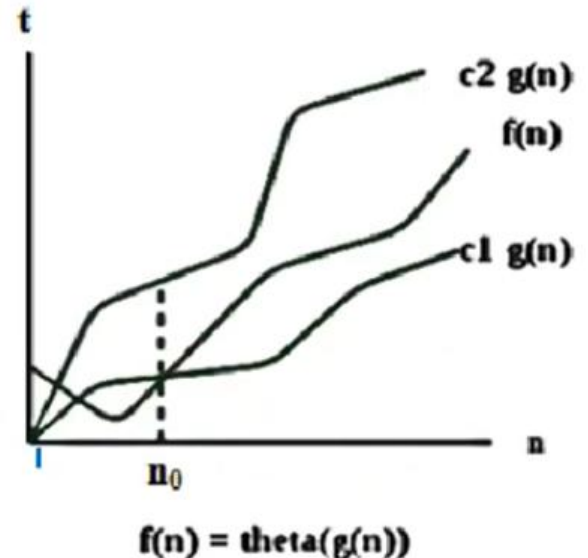
$$0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

for all  $n \geq n_0\}$

- The above definition means, if  $f(n)$  is theta of  $g(n)$ , then the value  $f(n)$  is always between  $c_1 * g(n)$  and  $c_2 * g(n)$  for large values of  $n$  ( $n \geq n_0$ ).
- A simple way to get Theta notation of an expression is to drop low order terms and ignore leading constants.

$$2n^3 + 7n^2 + 2000 = \Theta(n^3)$$

**Dropping lower order terms is always Ok since there will always be a  $n_0$  after which  $\Theta(n^3)$  has higher values than  $\Theta(n^2)$  regardless of the**



## Time Efficiency – Asymptotic Notation

$$f(n) = \sqrt{5n^2 + 3n + 1}$$
$$\Theta(n^2)$$

10, 20, 6

$$(C_1) * n^2 \geq 5n^2 + 3n + 1$$

$$(C_2) * n^2 \leq 5n^2 + 3n + 1$$



4, 3, 2, 1

Any  
Questions ?

